

1. The Verification Argument

A task is not complete merely because the requested files changed. Completion is a claim about the relation between the requested outcome and the resulting state. Verification supplies evidence for that claim.

1. Codex can produce a plausible change that does not satisfy the intended behavior.
2. The final message is generated by the same system that made the change and is not independent evidence.
3. Observable checks can discriminate between at least some correct and incorrect states.
4. Therefore an end-to-end task should normally include checks that bear directly on the requested outcome.

The force of this argument depends on the task. Renaming a local variable may need only a relevant test or type check. Fixing a checkout flow may require unit tests, an integration path, and a browser reproduction. Verification should be proportional, not ceremonial.

2. Claims and Tests

Start with the claim. If the claim is "the parser now accepts escaped delimiters without changing existing syntax," a useful check must exercise escaped delimiters and existing cases. A generic linter says almost nothing about the behavior. If the claim is "the mobile modal no longer clips," a passing unit test cannot see the layout; rendered inspection can.

A verification plan is a mapping from claim to evidence, not a checklist copied from another task.

3. Reproduction Before Repair

When fixing a bug, first obtain an observable failure when practical. This establishes that the environment can express the problem and gives the eventual pass meaning. Without reproduction, a passing test after the patch may be a test that never covered the failure.

The strongest sequence is:

Observed failure -> minimal discriminating reproduction -> causal hypothesis -> change -> same reproduction passes -> relevant regression checks pass -> rendered or runtime behavior inspected when applicable.

4. Independent Evidence

Independence comes in degrees. A test written after the implementation can accidentally encode the implementation's mistake. A pre-existing failing test is stronger on that dimension. A second tool, a browser reproduction, or a read-only review can catch classes of error the implementation path did not.

Independence does not require a separate person for every task. It requires that the verification not simply restate the patch.

5. Verification Budget

OpenAI's Astra guidance recommends calibrating testing and avoiding repeated broad checks when they add no information. This guards against a different failure: verification theater. Re-running the same expensive suite after no relevant change creates cost without evidence.

A disciplined sequence is targeted first, broaden when consequence or changed surface justifies it, and repeat only after new changes or failures. For a one-line CSS correction, a visual check at affected breakpoints may dominate a full backend suite. For a shared type change, broad compilation may be essential.

6. Stop Conditions

Verification needs its own stop rule. Otherwise Astra's tendency to keep checking can turn a completed task into an unbounded QA expedition. Tell it what checks matter and when to return. Kundel's recommendation is explicit: state what finished looks like, what should be verified, and when the work should be handed back.

References: OpenAI, GPT-6 Astra model guidance; Dominik Kundel, "Five things I learned from using Astra"; OpenAI, Codex use cases.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 8: The Verification Argument

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>